Solution: Problem Challenge 2: Find the Smallest Missing Positive Number

Problem Statement

Given an unsorted array containing numbers, find the **smallest missing positive number** in it.

Note: Positive numbers start from '1'.

Example 1:

```
Input: [-3, 1, 5, 4, 2]
Output: 3
Explanation: The smallest missing positive number is '3'
```

Example 2:

```
Input: [3, -2, 0, 1, 2]
Output: 4
```

Example 3:

```
Input: [3, 2, 5, 1]
Output: 4
```

Example 4:

```
Input: [33, 37, 5]
Output: 1
```

Solution

This problem follows the **Cyclic Sort** pattern and shares similarities with `Find the Missing Number` with one big difference. In this problem, the numbers are not bound by any range so we can have any number in the input array.

However, we will follow a similar approach though as discussed in `Find the Missing Number` to place the numbers on their correct indices and ignore all numbers that are out of the range of the array (i.e., all negative numbers and all numbers greater than the length of the array). Once we are done with the cyclic sort we will iterate the array and the first index that does not have the correct number will be the smallest missing positive number!

Code

Here is what our algorithm will look like:

Java

```java
class Solution {

  // findNumber finds the smallest missing positive number in the array.
  public int findNumber(int[] nums) {
    int i = 0;
    while (i < nums.length) {
      if (nums[i] > 0 && nums[i] <= nums.length && nums[i] != nums[nums[i] - 1])
        swap(nums, i, nums[i] - 1);
      else
        i++;
    }

    // Find the first index where the element does not match its expected positive value
    for (i = 0; i < nums.length; i++)
      if (nums[i] != i + 1)
        return i + 1;

    // If all elements from 1 to nums.length are present, return nums.length + 1.
    return nums.length + 1;
  }
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm is O(n).

Space Complexity

The algorithm runs in constant space O(1).